Course 3: Language Modeling

Introduction

- A sequence of tokens $(w_1, w_2, \ldots, w_n)$
- For a position i, a language model (**LM**) predicts

$$P(w_i \,|(w_j)_{j \neq i}) \in \Delta^V$$

- In words: a LM predicts the probability of a token given its context

Introduction

*I went to the **???** yesterday*

P(*park* | *I went to the **???** yesterday*) = 0.1

P(*zoo* | *I went to the **???** yesterday*) = 0.07

...

P(*under* | *I went to the **???** yesterday*) = 0

Introduction

Why is it hard?

- **Large vocabularies**: 170,000 English words
- **Lots of possible contexts**:
  - For V possible tokens, there are V^L contexts of size L (in theory)
- **Inherent uncertainty**: not obvious even for humans

Contents

1. **Basic Approach**
   a. n-grams
   b. Neural modeling
2. **The Dense Embedding Paradigm**
   a. Word2Vec
   b. Recurrent neural networks
3. **Transformers**
   a. The original architecture
   b. Encoders
   c. Decoders

Basic Apporach

n-grams

Unigram

- Learn the *non-contextual* probability (=frequency) of each token:

$$P(w_i \,|(w_j)_{j \neq i}) = f$$

Example

chart against operations at influence the surface plays crown a inaro the three @ but the court lewis on hand american of seamen mu role due roger executives

n-grams

Bigram

- Predict based on the last token only:

$$P(w_i \,|(w_j)_{j \neq i}) = P_\theta(w_i | w_{i-1})$$

- (MLE): Measure next token frequency

Example

the antiquamen lost to dios nominated former is carved stone oak were problematic, 1910. his willingness to receive this may have been seen anything

n-grams

- Predict based on the n last tokens only:

$$P(w_i \,|(w_j)_{j \neq i}) = P_\theta(w_i | w_{i-n} \dots w_{i-1})$$

- (MLE): Measure occurences of tokens after $w_{i-n} \dots w_{i-1}$

Example (n=4)

eva gauthier performed large amounts of contemporary french music across the united states marshals service traveled to frankfurt, germany and took custody of the matthews

n-grams

✅ Easy to train
✅ Easy to interpret
✅ Fast inference

❌ Very limited context
❌ **Unable to extrapolate** : can only model what it has seen

Neural modeling

The most basic way to represent a discrete item (like a word) is **one-hot encoding**.

- Create a vector with a size equal to your vocabulary $|V|$.
- The vector is all zeros, except for a single '1' at the index for that specific word.

Example: For a vocabulary `{apple, king, queen, ...}`

- `apple` → `[1, 0, 0, 0, ...]`
- `king` → `[0, 1, 0, 0, ...]`
- `queen` → `[0, 0, 1, 0, ...]`

Neural modeling

This representation has a massive, built-in flaw: every single word vector is **orthogonal** (geometrically perpendicular) to every other.

The dot product (our simplest measure of similarity) is always zero:

$$\vec{w}_i \cdot \vec{w}_j = 0 \quad \text{for } i \neq j$$

- `similarity(king, queen)` = `[0,1,0] · [0,0,1]` = **0**
- `similarity(king, apple)` = `[0,1,0] · [1,0,0]` = **0**

To the model, **"king" is as unrelated to "queen" as it is to "apple"**. It has **zero** built-in concept of semantic similarity and cannot generalize.

Neural modeling

What if we use a neural network to learn the probability function $P(w_i \mid \text{context})$?

This could potentially:

1. Allow for a much richer, non-linear combination of context words.
2. Solve the sparsity problem by generalizing.

Neural modeling

Bengio et al., (2003) was a breakthrough, using a simple feed-forward network to predict the next word.

The Architecture:

1. Take the last `n` context words (e.g., "the cat sat on").
2. Convert each word to its **one-hot vector** (size $|V|$).
3. **Concatenate** these `n` vectors into one giant input vector.
4. Feed through a hidden layer and a `softmax` output layer.

Neural modeling

The network's final `softmax` layer outputs a probability distribution over the *entire* vocabulary. We need a way to measure how "wrong" this prediction is.

This is the job of a **loss function**. For classification tasks, we use **Cross-Entropy**.

1. **The "True" Distribution (y)**: This is the ground truth. It's just a one-hot vector where the correct next word has a probability of 1. e.g., for "sat": `y = [0, 0, 1, 0, ...]`
2. **The "Predicted" Distribution ($\hat{y}$)**: This is the `softmax` output from our model. e.g., `ŷ = [0.1, 0.05, **0.6**, 0.15, ...]`

Neural modeling

What properties this score should have?

- We only care about the probability assigned to the **one correct word**.
- If `p(correct_word)` is high (e.g., 0.99), the loss should be *very low*.
- If `p(correct_word)` is low (e.g., 0.01), the loss should be *very high*.

$$\text{Loss} = -\log(p_{\text{correct}})$$

Neural modeling

- **Confident & Correct**:
  - `p("sat") = 0.95`
  - `Loss = -log(0.95) ≈ 0.05` (Tiny loss!)
- **Unconfident & Correct**:
  - `p("sat") = 0.20`
  - `Loss = -log(0.20) ≈ 1.61` (Medium loss)
- **Confident & WRONG**:
  - The model predicted `p("jumped") = 0.9`, so `p("sat") = 0.01`
  - `Loss = -log(0.01) ≈ 4.60` (Massive loss!)

Neural modeling

This `-log(p)` score is a perfect measure of **surprise**.

- If `p` is high, the event was expected. Low surprise, low loss.
- If `p` is low, the event was a shock. High surprise, high loss.
- Training just means "adjust the model to be less surprised by the real data."

Neural modeling

So, what is "Cross-Entropy"?

It's the formal name for this concept. The *full* cross-entropy formula measures the difference between two distributions, y and $\hat{y}$:

$$H(y, \hat{y}) = -\sum_{i=1}^{|V|} y_i \log(\hat{y}_i)$$

- y_i is our **one-hot "truth" vector**. It's `0` for all words *except* the correct one (let's call its index c), where $y_c = 1$.
- The entire sum of $|V|$ terms gets multiplied by 0...
- ...except for the one single term for the correct word!

$$H(y, \hat{y}) = -(0 \cdot \log(\hat{y}_1) + \ldots + 1 \cdot \log(\hat{y}_c) + \ldots + 0 \cdot \log(\hat{y}_{|V|}))$$

Neural modeling

It simplifies to exactly what we started with:

$$\text{Loss} = -\log(\hat{y}_c)$$

In short: "Cross-Entropy" is the technically correct term, but for language modeling, it simplifies to the much more intuitive **Negative Log-Likelihood (NLL)**.

Neural modeling: side note

You might also see **Kullback-Leibler (KL) Divergence** mentioned, as it *also* measures the "distance" between the true distribution y and our prediction $\hat{y}$.

The Definitions (A Quick Reminder):

- **Cross-Entropy:** $H(y, \hat{y}) = -\sum_i y_i \log(\hat{y}_i)$
- **Entropy:** $H(y) = -\sum_i y_i \log(y_i)$
  (This measures the "surprise" or "uncertainty" of the true data itself)

Neural modeling: side note

The KL Divergence Formula:

KL Divergence is defined as the "extra surprise" from using $\hat{y}$ to represent y:

$$
\begin{aligned}
D_{KL}(y \,||\, \hat{y}) &= \sum_i y_i \log\left(\frac{y_i}{\hat{y}_i}\right) \\
&= \sum_i y_i (\log(y_i) - \log(\hat{y}_i)) \\
&= \sum_i y_i \log(y_i) - \sum_i y_i \log(\hat{y}_i) \\
&= \underbrace{(-H(y))}_{\text{from } \sum y_i \log(y_i)} - \underbrace{(-H(y, \hat{y}))}_{\text{from } \sum y_i \log(\hat{y}_i)} \\
&= H(y, \hat{y}) - H(y)
\end{aligned}
$$

Neural modeling: side note

$$H(y, \hat{y}) = D_{KL}(y \,||\, \hat{y}) + H(y)$$

- During training, our "truth" distribution y (the one-hot label) is **fixed and constant**.
- This means its entropy, $H(y)$, is also a **fixed constant**.
  (In fact, for a one-hot vector, the "truth" has no uncertainty, so its entropy is 0!)
- When we optimize a model, we are trying to find the minimum of the loss function. Adding a constant to a function **does not change where the minimum is**.
- Therefore, minimizing the Cross-Entropy is **mathematically identical** to minimizing the KL-Divergence.

The "limitation" is just that KL-Divergence includes an extra, useless-for-optimization term ($H(y)$). Cross-Entropy is simpler to calculate and achieves the *exact same training result*.

Neural modeling

Problem 1: The Dimensionality Explosion

The input layer is enormous. For a modest vocabulary of `|V| = 50,000` and a context of `n=4` words:

$$\text{Input Size} = n \times |V| = 4 \times 50,000 = \mathbf{200{,}000}$$

This is computationally incredibly inefficient.

Neural modeling

Problem 2: No Shared Knowledge

Context: "the cat sat" (n=3)

`|V|`: `50,000`

`v_the`: `[0, 0, ... 1, ... 0, 0]` (50k dims)

`v_cat`: `[0, 1, ... 0, ... 0, 0]` (50k dims)

`v_sat`: `[0, 0, ... 0, ... 1, 0]` (50k dims)

Final Input: `[v_the | v_cat | v_sat]` (a single 150,000-dim vector)

Neural modeling

The first hidden layer is a giant weight matrix `W` of size `(150,000 x d)`.

The weights that would learn the concept of "cat" in position 2 have no relationship to the weights that would learn the concept of "cat" in position 3.

It has *NO PARAMETER SHARING*!

Neural modeling

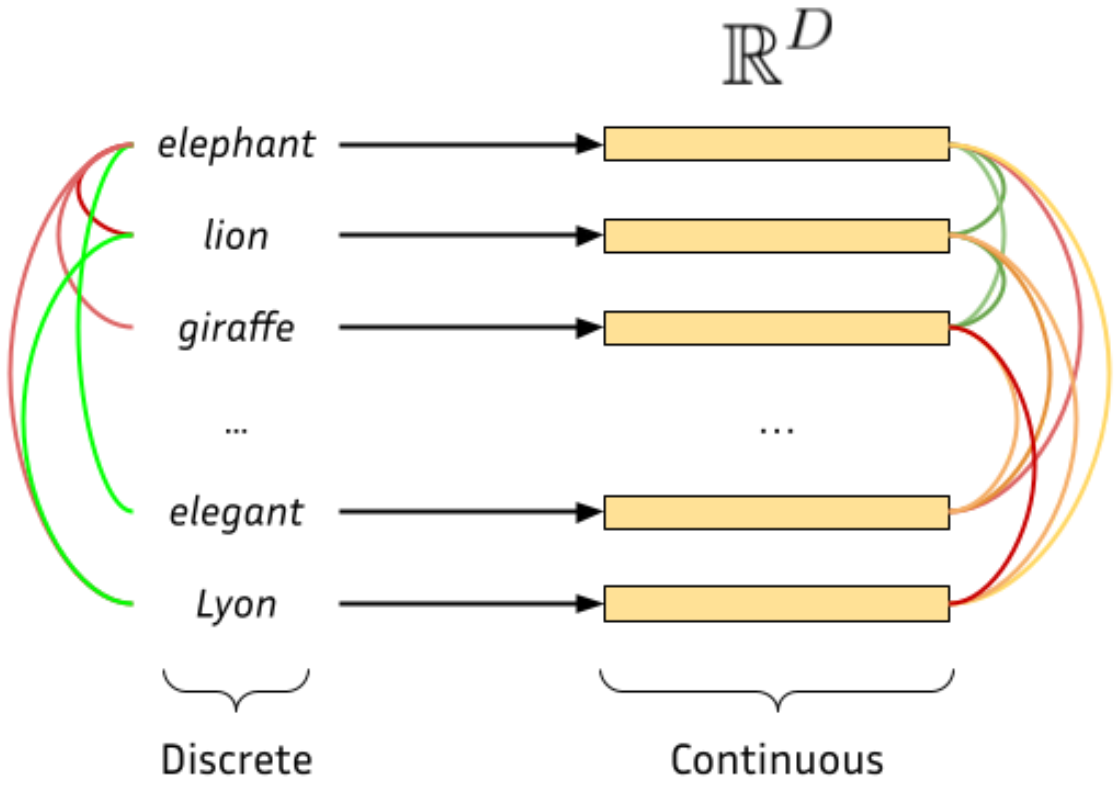
The fundamental flaw isn't the network architecture or the loss function, it's the **input encoding**.

We need a way to represent words that is:

1. **Dense** (not sparse and full of zeros).
2. **Low-Dimensional** (e.g., 300 dimensions, not 50,000).
3. **Semantic** (words with similar meanings should have similar vectors).

The Dense Embedding Paradigm

Word2Vec


$\mathbb{R}^D$
elephant
lion
giraffe
...
...
elegant
Lyon
Discrete
Continuous

Word2Vec

An `Embedding Layer` is simply a **giant lookup table**.

- It's one **single matrix**, let's call it `E`.
- **Rows:** One row for every word in your vocabulary (`|V|`).
- **Columns:** The number of dimensions you want your embedding to have (e.g., `d=300`).

$$E = \begin{bmatrix} \leftarrow & \text{vector for "a"} & \rightarrow \\ \leftarrow & \text{vector for "apple"} & \rightarrow \\ \vdots & \vdots & \vdots \\ \leftarrow & \text{vector for "cat"} & \rightarrow \\ \vdots & \vdots & \vdots \\ \leftarrow & \text{vector for "zebra"} & \rightarrow \end{bmatrix} \quad (\text{A } |V| \times d \text{ matrix})$$

Word2Vec

Mathematically, the **"lookup" is just a matrix multiplication** with a one-hot vector.

- `v_cat` (one-hot): `[0, 0, ..., 1, 0, ...]` (at index 500)
- `E` (Embedding Matrix): `[|V| x 300]`

$$\underbrace{[0, 0, \ldots, 1, \ldots, 0]}_{1 \times |V|} \quad @ \quad \underbrace{\begin{bmatrix} \vdots \\ \leftarrow \text{vector } 500 \rightarrow \\ \vdots \end{bmatrix}}_{|V| \times 300} \quad = \quad \underbrace{[\leftarrow \text{vector } 500 \rightarrow]}_{1 \times 300}$$

Word2Vec

In code, we *never* create the giant one-hot vectors. It's a massive waste.

We just pass **integer indices** directly to the layer.

1. **Input:** A 2D tensor of integers `[batch_size, sequence_length]`.

```
# Batch 1: "the cat sat"
# Batch 2: "the dog and"
X = [
      [ 4, 500, 512],
      [ 4,  50,   6]
    ]
# Shape: [2, 3]
```

Word2Vec

2. **Operation:** The layer performs an efficient `lookup`. It replaces each integer with its corresponding vector (row) from the `E` matrix.
3. **Output:** A 3D tensor `[batch_size, sequence_length, embedding_dim]`.

```
# Output:
[
  [ [v_the], [v_cat], [v_sat] ],  # Batch 1
  [ [v_the], [v_dog], [v_and] ]   # Batch 2
]
# Shape: [2, 3, 300]
```

Word2Vec

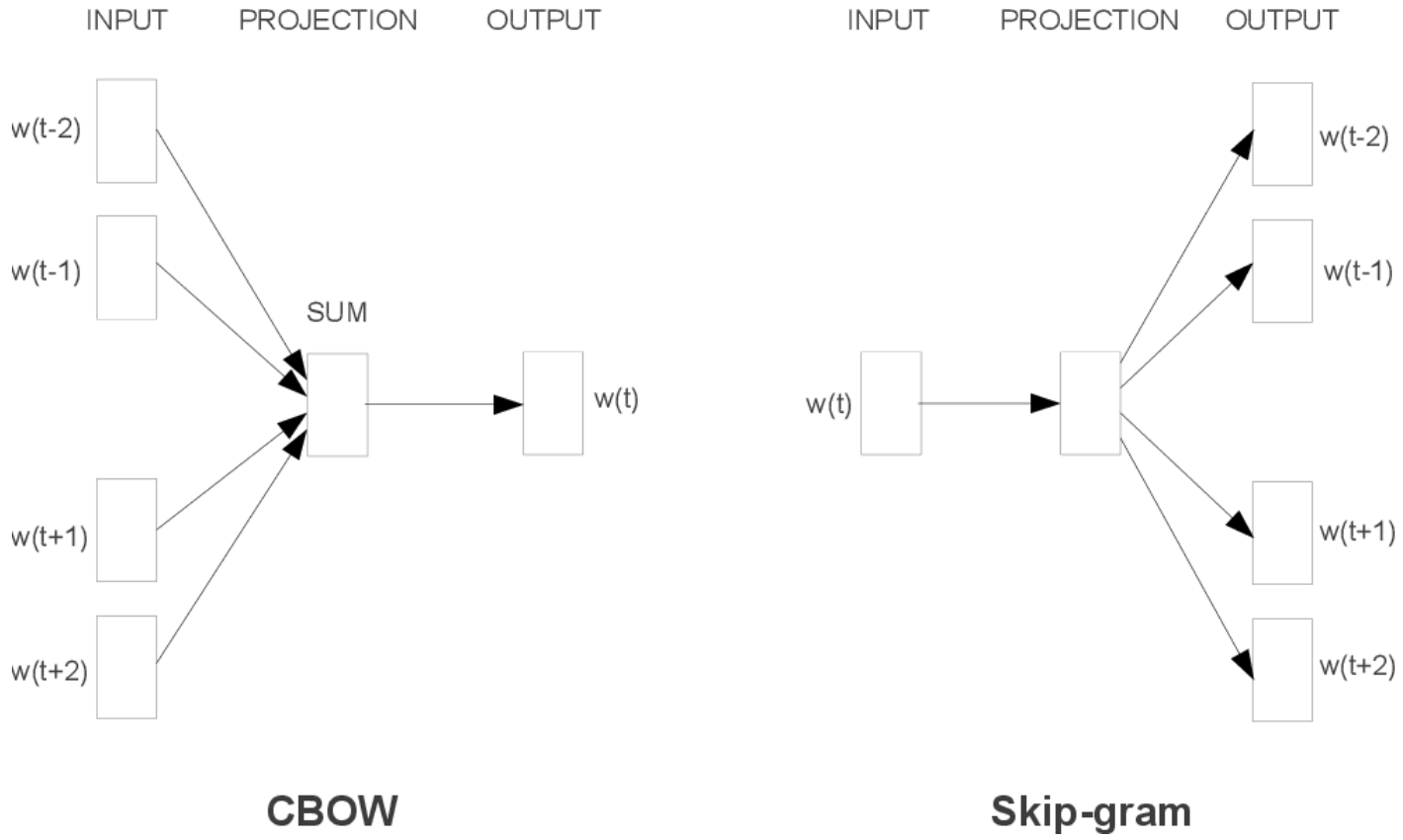
This **"lookup"** architecture is the **key to sharing parameters**.

- The vector `v_the` (which is row `E[4]`) is retrieved for **(Batch 0, Pos 0)** and **(Batch 1, Pos 0)**.
- During backpropagation, the "error" (gradient) from *both* of these positions will flow back and be **summed** to update that **one single row** `E[4]`.

The model learns **one** vector for "the", not a separate vector for "the at position 0".

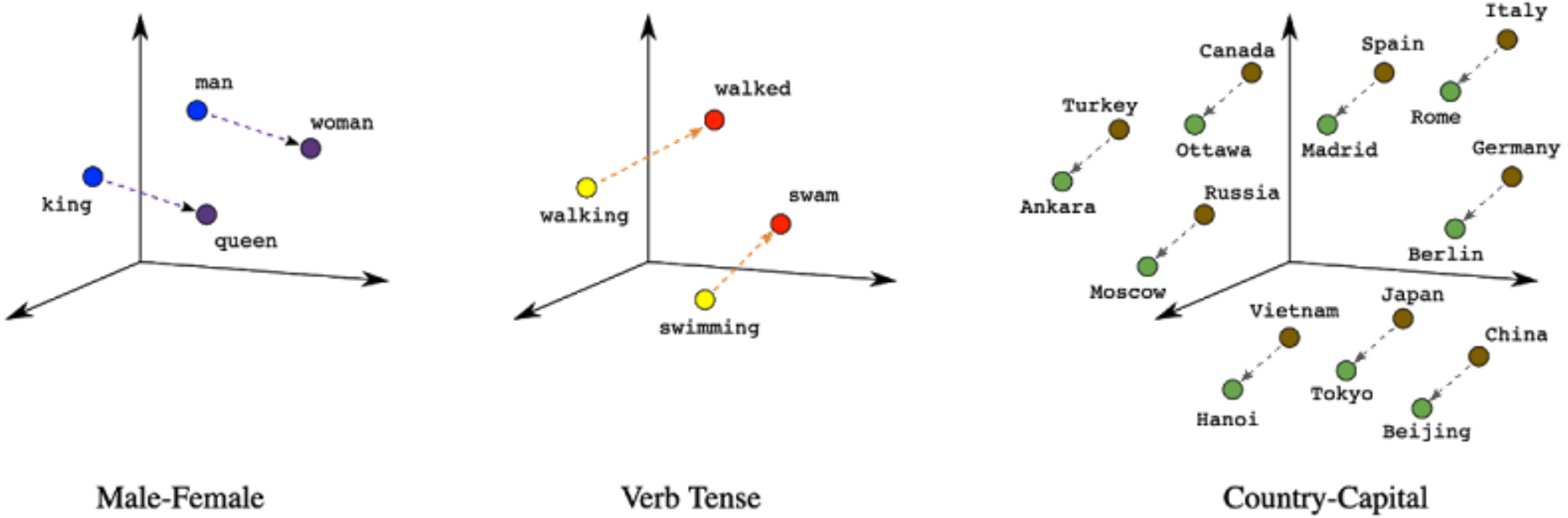
Word2Vec

Mikolov et al. (2013)



Word2Vec

Vector directions are now related to **concepts** (some more intepretable than others).



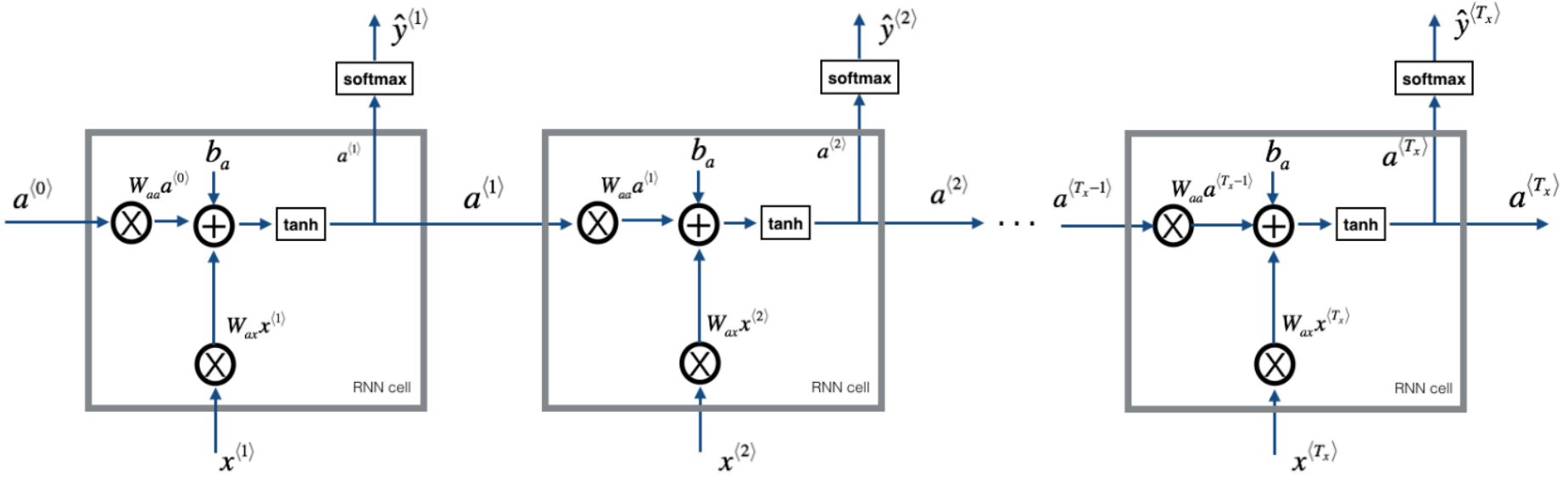
Male-Female Verb Tense Country-Capital

Word2Vec

- It solves the `king - man + woman ≈ queen` analogy.
- It knows "cat" and "kitten" are close.
- It solves the generalization problems in some ways.

But it has a huge limitation:
It is a *"positionless bag of context"*: the meaning of "The cat sat on the mat" and "The mat sat on the cat" is lost (more with CBOW than with skip-gram).

Recurrent neural networks



- $(x^{<1>}, \ldots, x^{<n>})$: training sequence
- Learnable matrices $U^{<t>}$ and $W^{<t>}$, $1 \leq t \leq n$
- θ: parameters of the RNN
- Cross-entropy loss $\mathcal{L}_{ce}$:

Recurrent neural networks

The goal is to learn matrices U and W in each cell t such that

$$h_t = \tanh(U^{<t>} h^{<t-1>} + W^{<t>} x^{<t>} + b^{<t>})$$

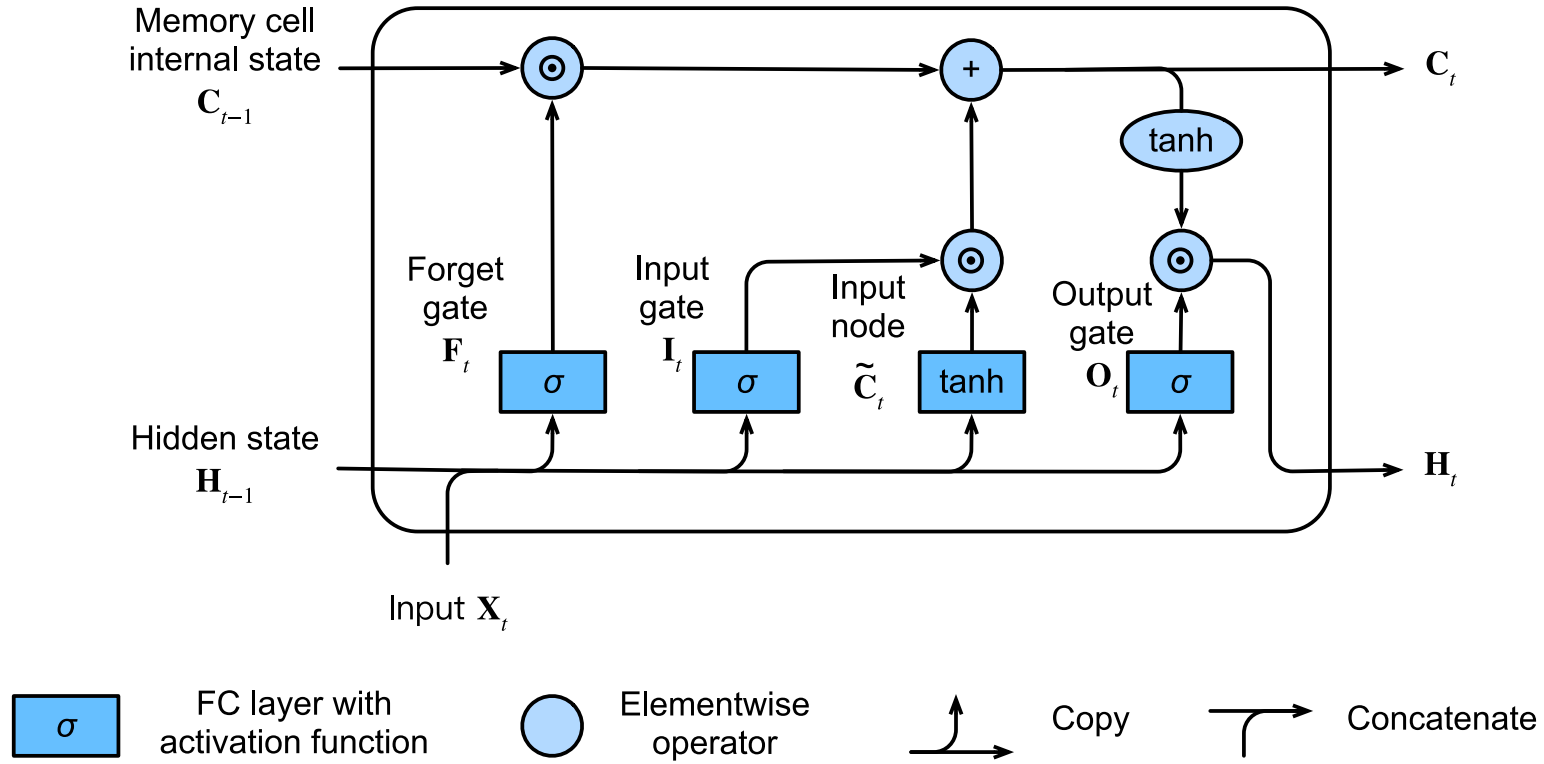
For language modeling, we use a $\mathrm{softmax}$ on top of the hidden state and compute the cross-entropy loss

$$\mathcal{L}_{ce}(x, \theta) = -\sum_{i=2}^{t} \mathbb{1}_{w_i} \cdot \log P_\theta(x^{<t>} | x^{<t-1>}, h^{<t-1>})$$

Recurrent neural networks

- Strenghts
  - **Can extrapolate** (works with continuous features)
- Limitations
  - **Context dilution** when information is far away: the model can only write on the bandwidth

Recurrent neural networks



- $\tanh$ (range $[-1, 1]$) is used to decide how to store information (the content).
- sigmoid (range $[0, 1]$) is used to decide how much information to store (the "gate").

Recurrent neural networks

$$\text{Forget gate: } f^{<t>} = \sigma(U_f^{<t>} h^{<t-1>} + W_f^{<t>} x^{<t>} + b_f^{<t>})$$

$$\text{Input gate: } i^{<t>} = \sigma(U_i^{<t>} h^{<t-1>} + W_i^{<t>} x^{<t>} + b_i^{<t>})$$

$$\text{Output gate: } o^{<t>} = \sigma(U_o^{<t>} h^{<t-1>} + W_o^{<t>} x^{<t>} + b_o^{<t>})$$

$$\text{Candidate state: } S^{<t>} = \tanh(U_S^{<t>} h^{<t-1>} + W_S^{<t>} x^{<t>} + b_S^{<t>})$$

$$\text{Cell state: } C^{<t>} = C^{<t-1>} \odot f^{<t>} + S_t \odot i^{<t>}$$

$$\text{Hidden state: } h^{<t>} = \tanh(C^{<t>}) \odot o^{<t>}$$

Recurrent neural networks

RNNs

✅ **Causal** modeling, no more bag of context.
❌ **Vanishing gradient** for long sequences => poor long term memory
❌ **Slow** to train. You cannot calculate step t until you have finished step $t-1$.

LSTMs

✅ Cell state is like a **bandwidth** where each cell can write/delete information.
✅ When the **gradient flows backward**, it flows primarily **through element-wise multiplications and additions**: much **"cleaner" and more stable** than the nested $\tanh$ of a vanilla RNN.
❌ **A lot of gates** and moving parts. This **led to the GRU** (Gated Recurrent Unit), which is **a simplified version** that often **works just as well**.
❌ Still **slow** to train.

Transformers

The original architecture

Information flow in RNN

How many steps between source of info and current position?

- *What is the previous word?* => $\mathcal{O}(L)$
- *What is the subject of verb X?* => $\mathcal{O}(L)$
- *What are the other occurences of current word?* => $\mathcal{O}(L^2)$
- ...

The original architecture

Information flow in transformers

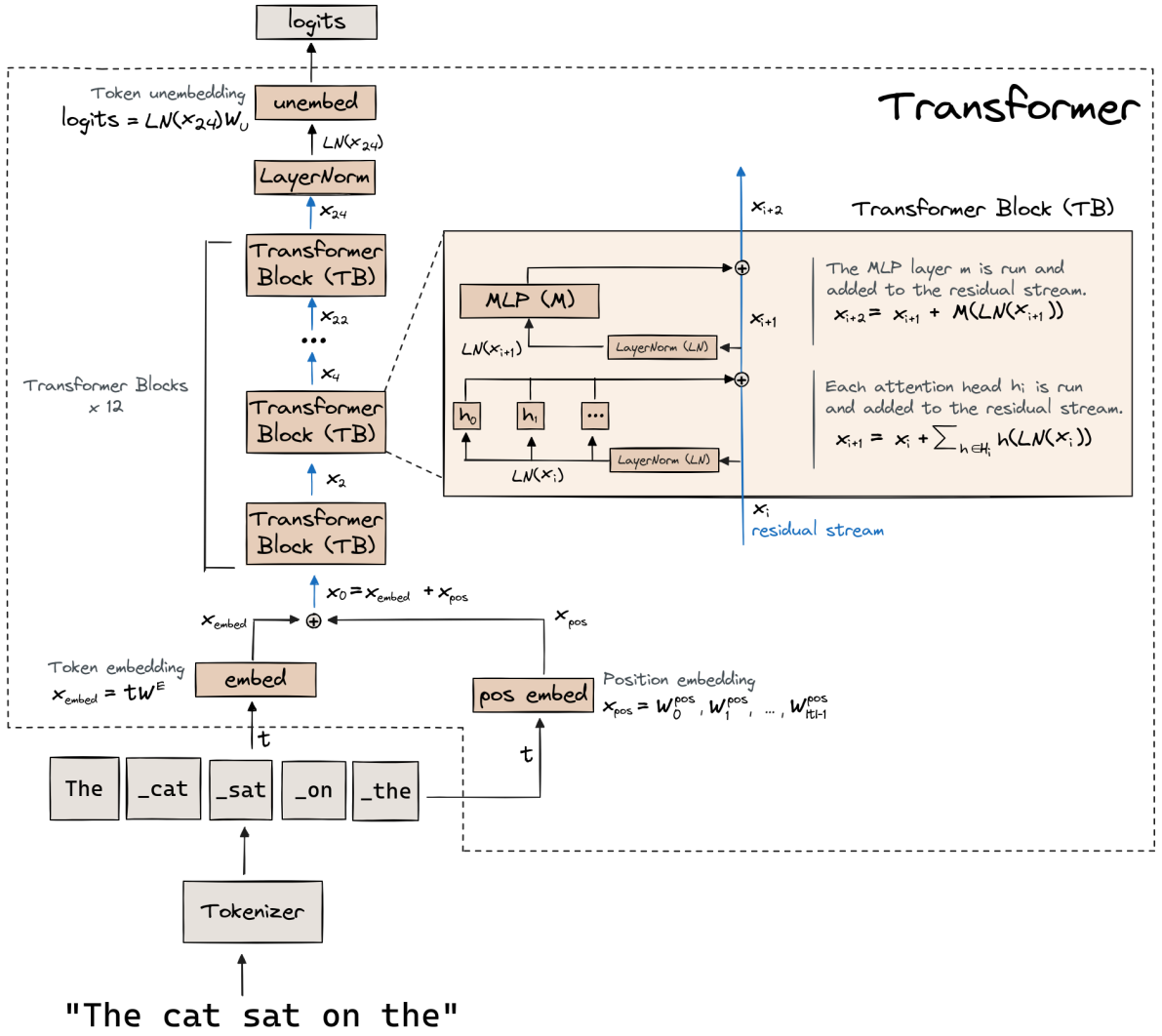
How many steps between source of info and current position?

- *What is the previous word?* => $\mathcal{O}(1)$
- *What is the subject of verb X?* => $\mathcal{O}(1)$
- *What are the other occurences of current word?* => $\mathcal{O}(1)$
- ... => $O(1)$

The original architecture

- A Transformer network T_θ
- Input: Sequence of vectors $(e_1, \ldots, e_n) \in \mathbb{R}^D$
- Output: Sequence of vectors $(h_1, \ldots, h_n) \in \mathbb{R}^D$
- Each h_i may depend on the whole input sequence $(e_1, \ldots, e_n)$

The original architecture



The original architecture

Before going in the network:

- Given an input token sequence $(w_1, \ldots, w_n)$
- We retrieve token embeddings $(e_{\text{token}}(w_1), \ldots, e_{\text{token}}(w_n)) \in \mathbb{R}^D$
- We retrieve position embeddings $(e_{\text{pos}}(1), \ldots, e_{\text{pos}}(n)) \in \mathbb{R}^D$
- We compute input embeddings: $e_{\text{input}} = e_{\text{token}}(w_i) + e_{\text{pos}}(i)$

The original architecture

Attention Mechanism: The Foundation

Self-attention **processes tokens as a set, not a sequence**. It is **permutation-invariant**, $\text{attn}(\{A, B, C\})$ gives the same *set* of outputs as $\text{attn}(\{C, B, A\})$.

With three tokens embeddings $x_1, x_2, x_3 \in \mathbb{R}^D$, the attention mechanism computes:

$$\text{attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q = XW_Q$, $K = XW_K$, and $V = XW_V$ are **learned linear projections**.

But **language is not a set of words**: "The dog chased the cat" $\neq$ "The cat chased the dog"

The original architecture

Attention mechanism: Queries and Keys

Step 1: Compute interaction scores

Given input matrix $X \in \mathbb{R}^{L \times D}$ where L is sequence length:

$$Q = XW_Q \in \mathbb{R}^{L \times d_k}, \quad K = XW_K \in \mathbb{R}^{L \times d_k}$$

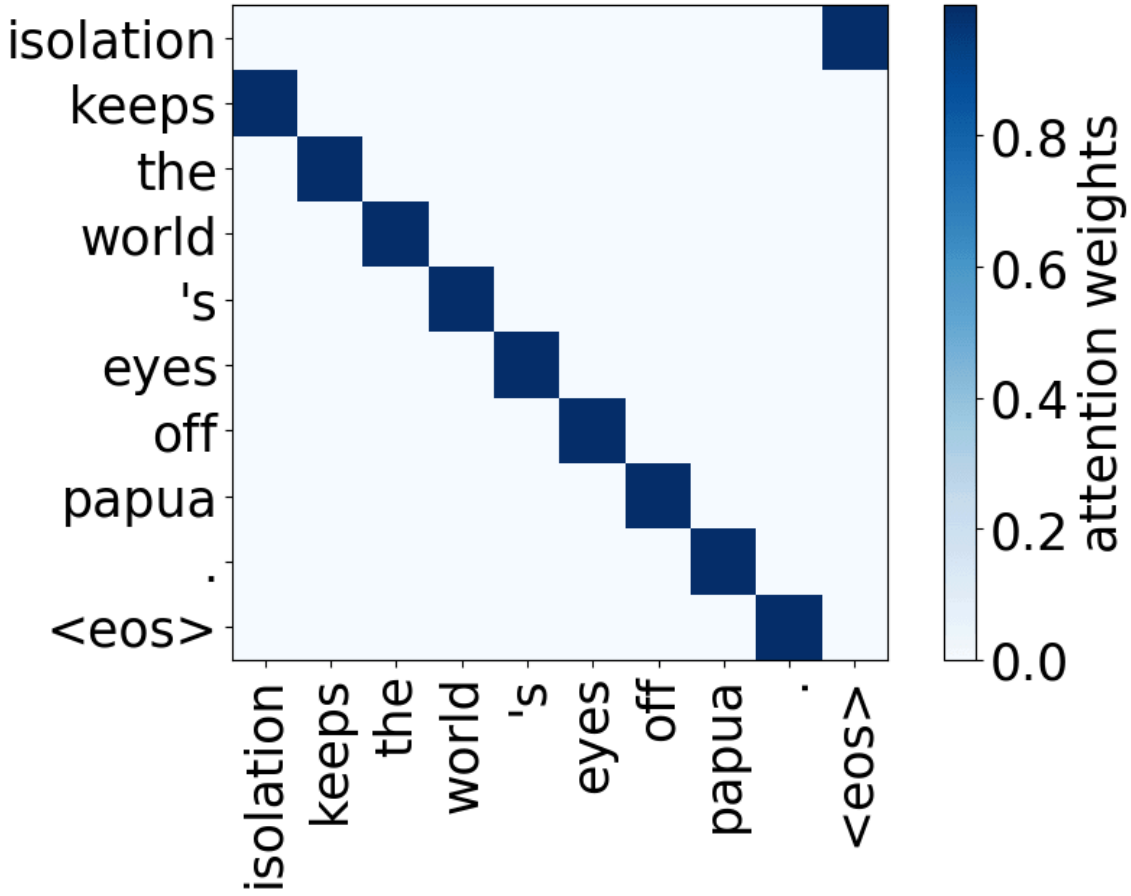
The scores matrix is $S = QK^T \in \mathbb{R}^{L \times L}$. Entry $S_{i,j}$ measures how much token i should attend to token j.

$$S_{i,j} = Q_i \cdot K_j = \sum_{d=1}^{d_k} Q_{i,d} \cdot K_{j,d}$$

This is a dot product measuring similarity between query i and key j.

The original architecture

Attention mechanism: Queries and Keys



The original architecture

The Scaling Factor: Why $\sqrt{d_k}$?

Assume $Q_{i,d}$ and $K_{j,d}$ are independent random variables with mean 0 and variance 1. Then:

$$\mathbb{E}[S_{i,j}] = \mathbb{E}\left[\sum_{d=1}^{d_k} Q_{i,d}K_{j,d}\right] = 0$$

$$\mathrm{Var}(S_{i,j}) = \mathrm{Var}\left(\sum_{d=1}^{d_k} Q_{i,d}K_{j,d}\right) = d_k$$

The **variance scales linearly with dimension**. Large dot products **push softmax into saturation regions** where **gradients vanish**.

Solution: Scale by $\sqrt{d_k}$ to normalize variance back to 1:

$$\mathrm{Var}\left(\frac{S_{i,j}}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$$

The original architecture

Values: Gathering Information

$$V = XW_V \in \mathbb{R}^{L \times d_v}$$

The output is a weighted combination:

$$\text{Output} = AV \in \mathbb{R}^{L \times d_v}$$

For position i:

$$\text{Output}_i = \sum_{j=1}^{L} A_{i,j} V_j$$

The output at position i is a mixture of value vectors from all positions, weighted by attention scores.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The original architecture

Sentence: "cat sat mat" with $D = 4$, $d_k = d_v = 2$

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \quad W_Q = W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \\ 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

Queries and Keys:

$$Q = K = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$$

The original architecture

Scores: $S = QK^T = \begin{bmatrix} 4 & 0 & 2 \\ 0 & 4 & 2 \\ 2 & 2 & 2 \end{bmatrix}$

With $\sqrt{d_k} = \sqrt{2} \approx 1.41$: $S/\sqrt{d_k} \approx \begin{bmatrix} 2.83 & 0 & 1.41 \\ 0 & 2.83 & 1.41 \\ 1.41 & 1.41 & 1.41 \end{bmatrix}$

Attention weights after softmax:

$$A \approx \begin{bmatrix} 0.78 & 0.05 & 0.20 \\ 0.05 & 0.78 & 0.20 \\ 0.33 & 0.33 & 0.33 \end{bmatrix}$$

The original architecture

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \\ 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad A \approx \begin{bmatrix} 0.78 & 0.05 & 0.20 \\ 0.05 & 0.78 & 0.20 \\ 0.33 & 0.33 & 0.33 \end{bmatrix}$$

$$V = XW_V = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 0.5 & 0.5 \end{bmatrix}, \quad AV = \begin{bmatrix} 0.88 & 0.15 \\ 0.15 & 0.88 \\ 0.495 & 0.495 \end{bmatrix}$$

The original architecture

Multi-Head Attention: Multiple Perspectives

Single attention learns one notion of similarity.

Solution: Run multiple attention heads in parallel, each with different learned projections.

For h heads with head dimension $d_k = D/h$:

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$$

Each head has its own weight matrices $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{D \times d_k}$.

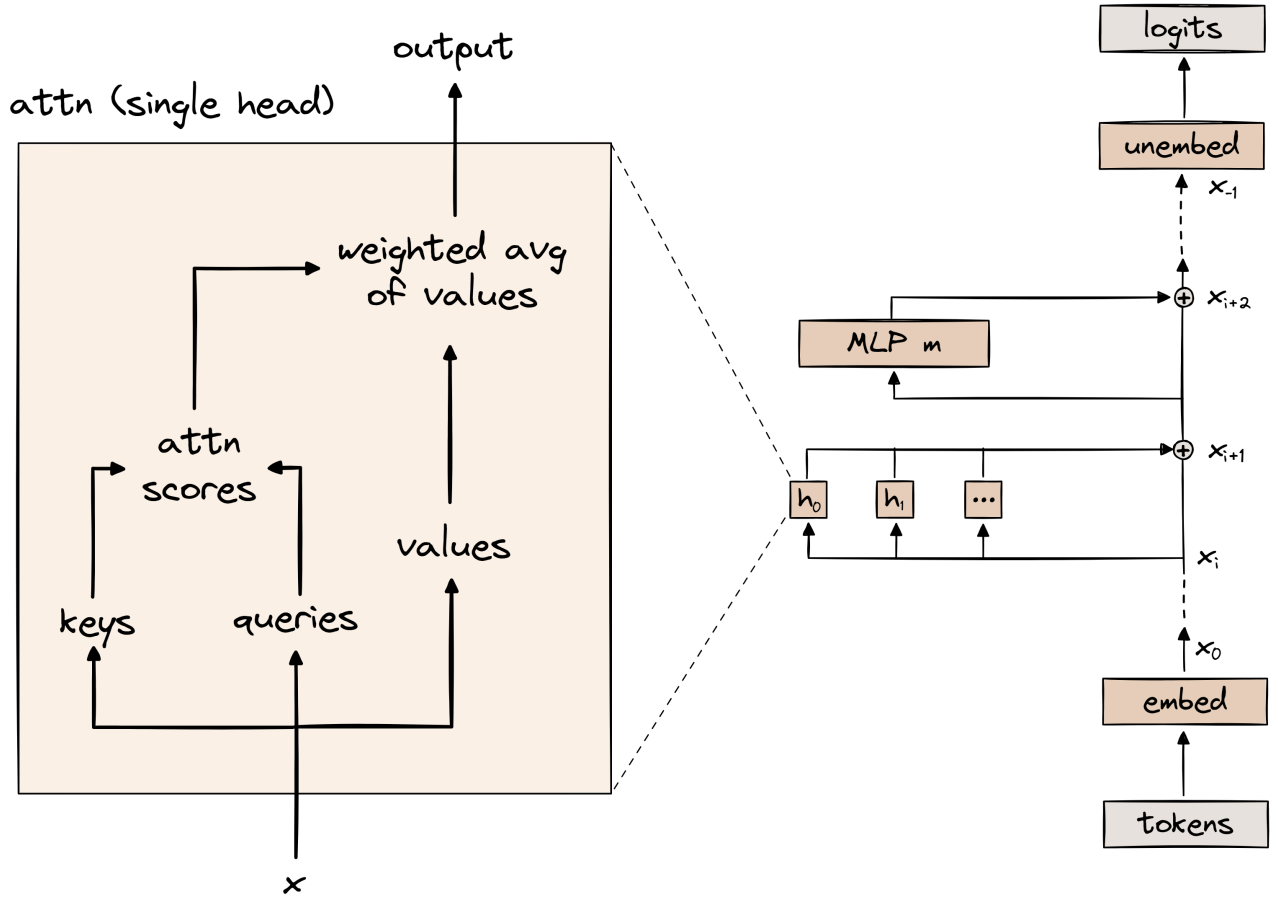
Concatenate outputs and project:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

where $W_O \in \mathbb{R}^{D \times D}$.

The original architecture

Multi-Head Attention: Multiple Perspectives



The original architecture

Modern flavors : Grouped-Query Attention

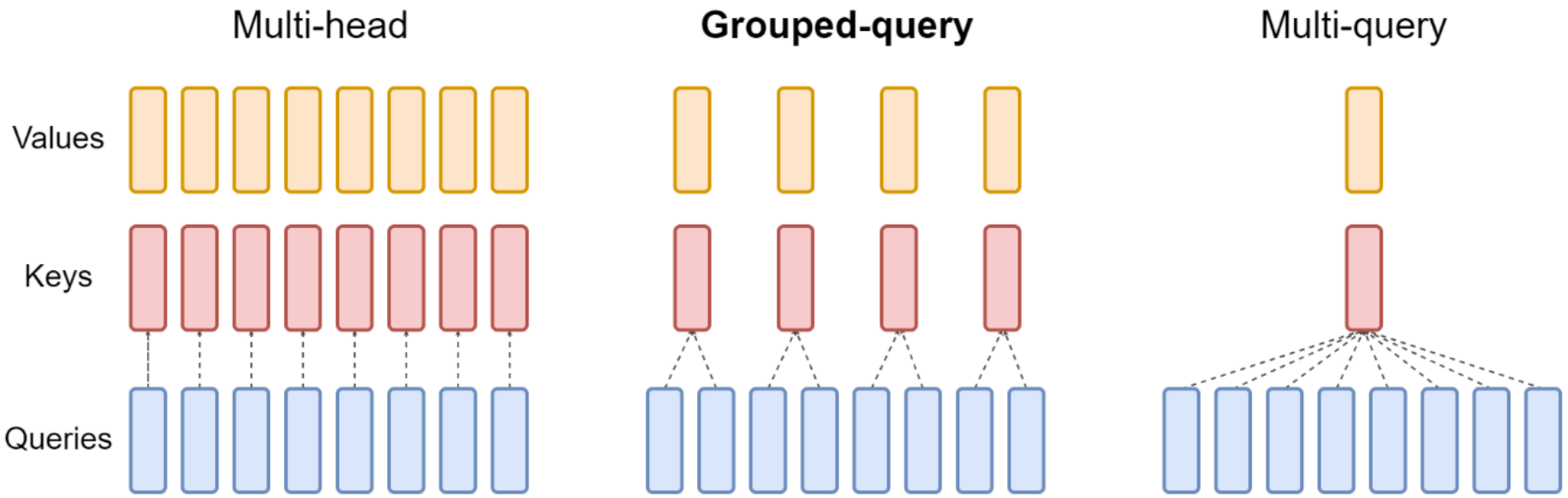


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

The original architecture

The Position Problem

If we permute rows of X, we permute rows of Q, K, V. **The attention operation sees inputs as a set**.

$$X_1 = \begin{bmatrix} \text{cat} \\ \text{chased} \\ \text{dog} \end{bmatrix}, \quad X_2 = \begin{bmatrix} \text{dog} \\ \text{chased} \\ \text{cat} \end{bmatrix}$$

The attention mechanism produces the same set of output vectors, just reordered.

⚠️ **RNNs didn't have this problem:** $h_t = f(h_{t-1}, x_t)$ bakes in position through sequential processing. Transformers trade this for parallelism.

The original architecture

Positional encoding

$$e_{\text{input}}(w_i) = e_{\text{token}}(w_i) + e_{\text{pos}}(i)$$

where $e_{\text{token}}(w_i) \in \mathbb{R}^D$ is the token embedding and $e_{\text{pos}}(i) \in \mathbb{R}^D$ is the positional encoding.

How do we define $e_{\text{pos}}(i)$?

1. Learned absolute positional embeddings (BERT, GPT-2/3)
2. Fixed sinusoidal positional encodings (original Transformer)
3. Relative positional encodings (RoPE, ALiBi)

The original architecture

Strategy A: Learned Absolute PE

The simplest approach: treat position as another vocabulary.

Create an embedding matrix $E_{\text{pos}} \in \mathbb{R}^{L_{\max} \times D}$ where $L_{\max}$ is the maximum sequence length.

$$e_{\text{pos}}(i) = E_{\text{pos}}[i]$$

These **vectors are initialized randomly** and **trained via backpropagation**, exactly like token embeddings.

✅ The model learns whatever positional representation works best for the task. Simple to implement.
❌ Cannot handle sequences longer than $L_{\max}$ seen during training.

The original architecture

Strategy B: Sinusoidal Absolute PE

Original approach from *Attention Is All You Need*. Use fixed mathematical functions that encode position through oscillations.

For position pos and dimension index $k \in \{0, 1, \ldots, D/2 - 1\}$:

$$\text{PE}_{(\text{pos},2k)} = \sin\left(\frac{\text{pos}}{10000^{2k/D}}\right)$$

$$\text{PE}_{(\text{pos},2k+1)} = \cos\left(\frac{\text{pos}}{10000^{2k/D}}\right)$$

Each dimension pair oscillates at a different frequency. **Low indices (k small) oscillate fast, high indices oscillate slowly**.

The original architecture

Strategy B: Sinusoidal Absolute PE

Example with $D = 4$ at position 0:

$$\text{PE}(0) = [\sin(0/10000^0), \cos(0/10000^0), \sin(0/10000^{2/4}), \cos(0/10000^{2/4})] = [0, 1, 0, 1]$$

At position 1:

$$\text{PE}(1) = [\sin(1), \cos(1), \sin(1/100), \cos(1/100)] \approx [0.841, 0.540, 0.010, 0.999]$$

Each position gets a unique fingerprint of sine/cosine values.

The original architecture

Strategy B: Sinusoidal Absolute PE

Recall that attention computes $Q_i \cdot K_j$ where $Q_i = W_Q(e_{\text{token}}(w_i) + \text{PE}(i))$ and $K_j = W_K(e_{\text{token}}(w_j) + \text{PE}(j))$.

Expanding the dot product:

$$\begin{aligned} S_{i,j} &= Q_i \cdot K_j \\ &= W_Q(e_{\text{token}}(w_i) + \text{PE}(i)) \cdot W_K(e_{\text{token}}(w_j) + \text{PE}(j)) \\ &= W_Q \cdot e_{\text{token}}(w_i) \cdot W_K \cdot e_{\text{token}}(w_j) \quad (\text{content-content}) \\ &+ W_Q \cdot e_{\text{token}}(w_i) \cdot W_K \cdot \text{PE}(j) \quad (\text{content-position}) \\ &+ W_Q \cdot \text{PE}(i) \cdot W_K \cdot e_{\text{token}}(w_j) \quad (\text{position-content}) \\ &+ W_Q \cdot \text{PE}(i) \cdot W_K \cdot \text{PE}(j) \quad (\text{position-position}) \end{aligned}$$

When we compute $e_{\text{input}} = e_{\text{token}} + \text{PE}$, we create **composite vectors** that enter attention. The **linearity of addition and matrix multiplication preserves geometric structure**.

The original architecture

Strategy C: Relative Positional Encoding

Absolute PE adds position once at input. Relative PE injects position directly into each attention computation.

$$\text{Score}_{i,j} = Q_i \cdot K_j + \text{bias}_{i,j}$$

where $\text{bias}_{i,j}$ is a function of $i - j$, the relative offset.

1. Rotary Position Embeddings (RoPE): Apply rotation matrices to queries and keys: The score depends on relative position $j - i$ through rotation angle. LLaMA, PaLM.
2. Attention with Linear Biases (ALiBi): Add a learned linear penalty: $\text{bias}_{i,j} = -m \cdot |i - j|$, where $m > 0$ is head-specific. Tokens farther apart get lower scores. Used in BLOOM.

The original architecture

Strategy C: Relative Positional Encoding

Absolute PE (learned or sinusoidal) is added once at the input. As information flows through many transformer layers, positional information can degrade.

✅ **Better length extrapolation:** Models trained on sequences of length 512 can handle length 2048+ at inference with relative PE.
✅ **Stronger inductive bias:** Language often exhibits local dependencies.

The original architecture

Bringing it all together. Given an input token sequence $(w_1, \ldots, w_n)$:

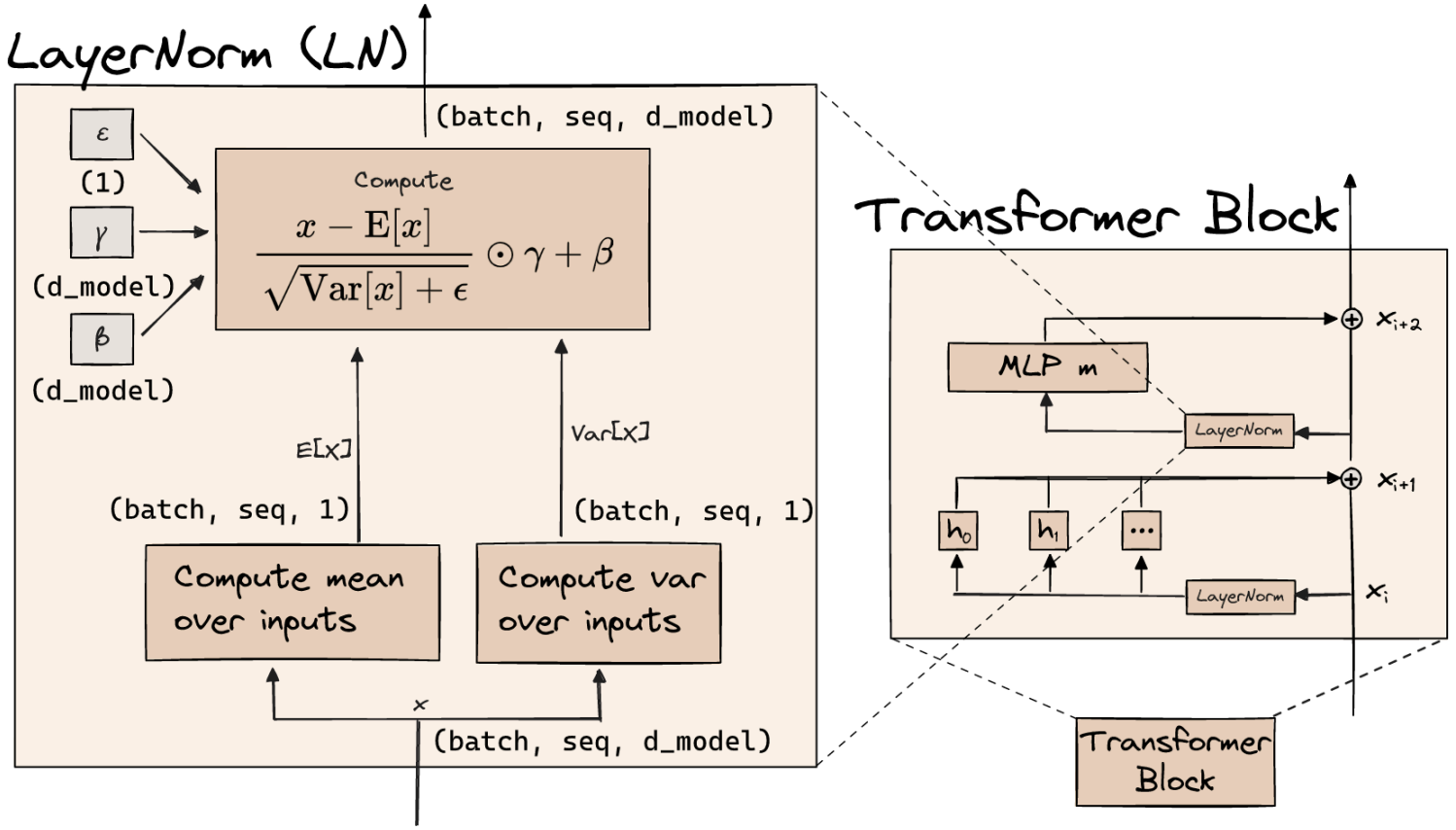
Step 1: Retrieve token embeddings from learned lookup table: $(e_{\text{token}}(w_1), \ldots, e_{\text{token}}(w_n)) \in \mathbb{R}^D$

Step 2: Generate or retrieve positional encodings: $(e_{\text{pos}}(1), \ldots, e_{\text{pos}}(n)) \in \mathbb{R}^D$. This uses learned embeddings (Strategy A), sinusoidal functions (Strategy B), or is handled separately in attention (Strategy C).

Step 3: Compute input embeddings: $e_{\text{input}}(w_i) = e_{\text{token}}(w_i) + e_{\text{pos}}(i)$; for relative PE, we skip this step and inject position during attention computation instead.

The original architecture

LayerNorm



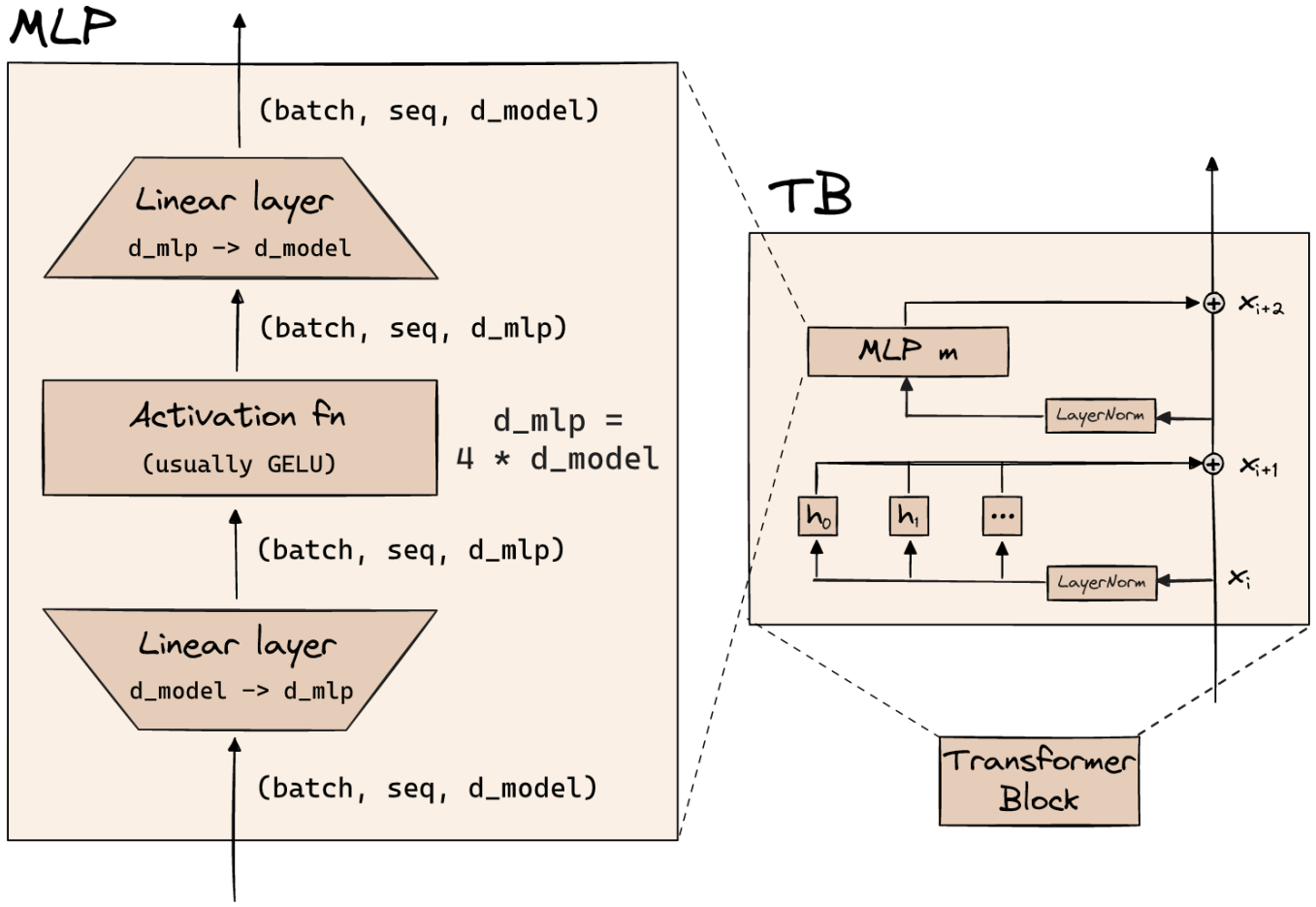
The original architecture

Modern flavors : RMSNorm

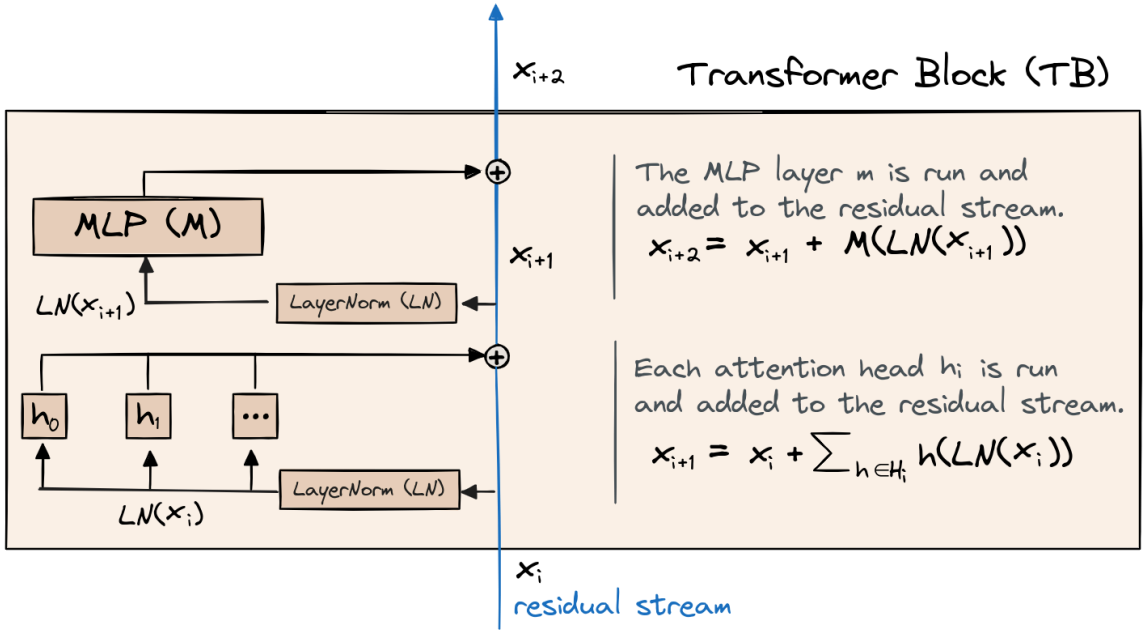
- Replaces LayerNorm
- Re-scaling is all you need

$$RMSNorm_g(a_i) = \frac{a_i}{\sqrt{\frac{1}{N}\sum_{j=1}^{N} a_j^2}} g_i$$

The original architecture

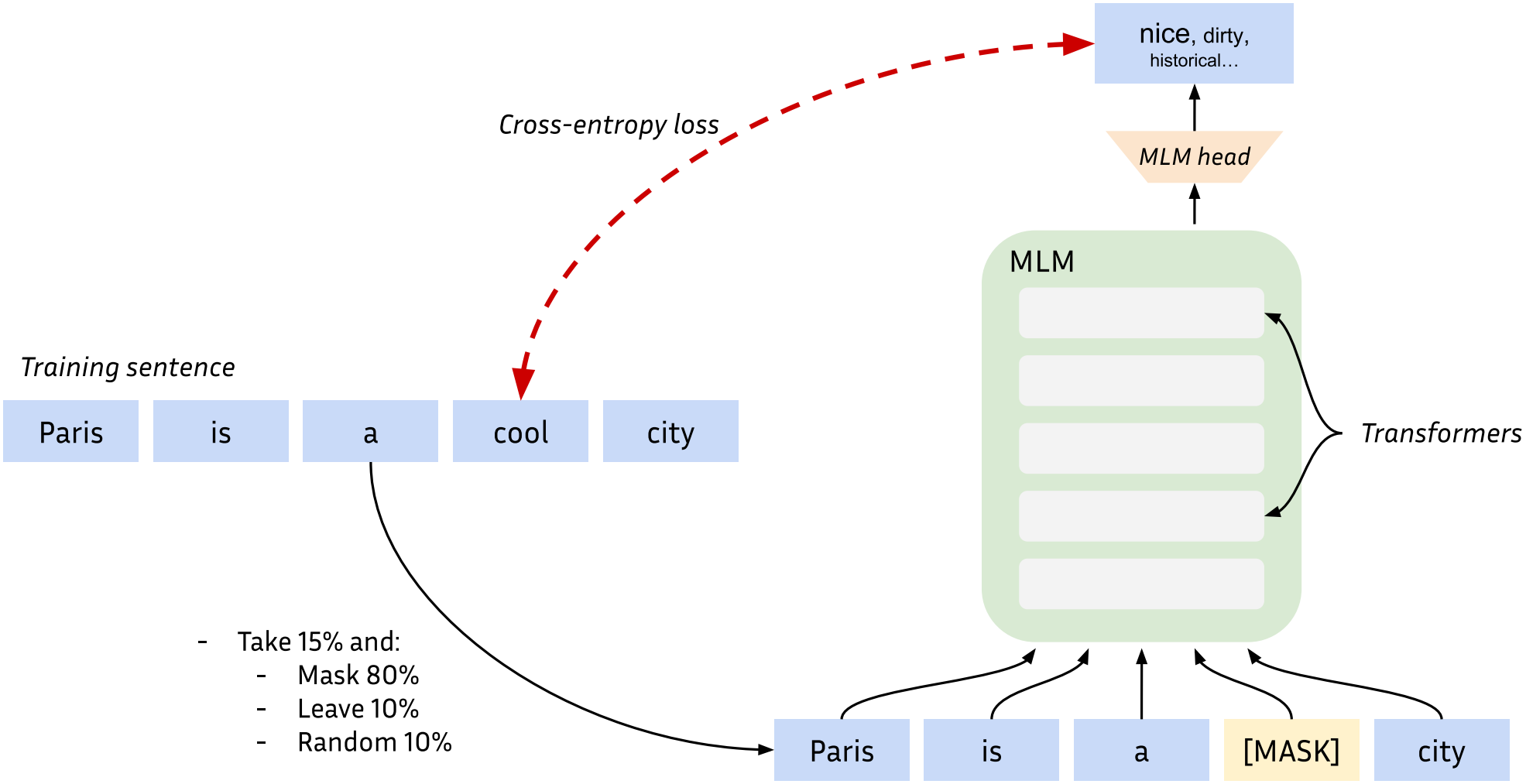


The original architecture



The **residual stream** act like a **bandwidth** and is **curcial in early training phase**, providing a **skip connection** for the gradient to flow when the weights are still random.

Encoders


nice, dirty, historical…
Cross-entropy loss
MLM head
MLM
Training sentence
Paris
is
a
cool
city
Transformers
- Take 15% and:
  - Mask 80%
  - Leave 10%
  - Random 10%
Paris
is
a
[MASK]
city

Encoders

BERT (Devlin et al., 2018)

- Pre-trained on 128B tokens from Wikipedia + BooksCorpus
- Additional Next Sentence Prediction (NSP) loss
- Two versions:
  - BERT-base (110M parameters)
  - BERT-large (350M parameters)
- **Cost**: ~1000 GPU hours

Encoders

RoBERTa (Liu et al., 2019)

- Pre-trained on ~~128B~~ **2T** tokens from web data (BERT x10)
- **No more** Next Sentence Prediction (NSP) loss
- Two versions:
  - RoBERTa-base (110M parameters)
  - RoBERTa-large (350M parameters)
- Better results in downstream tasks
- **Cost**: ~25000 GPU hours

Encoders

Multilingual BERT (mBERT)

- Pre-trained on 128B tokens from multilingual Wikipedia
- 104 languages
- One version:
  - mBERT-base (179M parameters)
- **Cost**: *unknown*

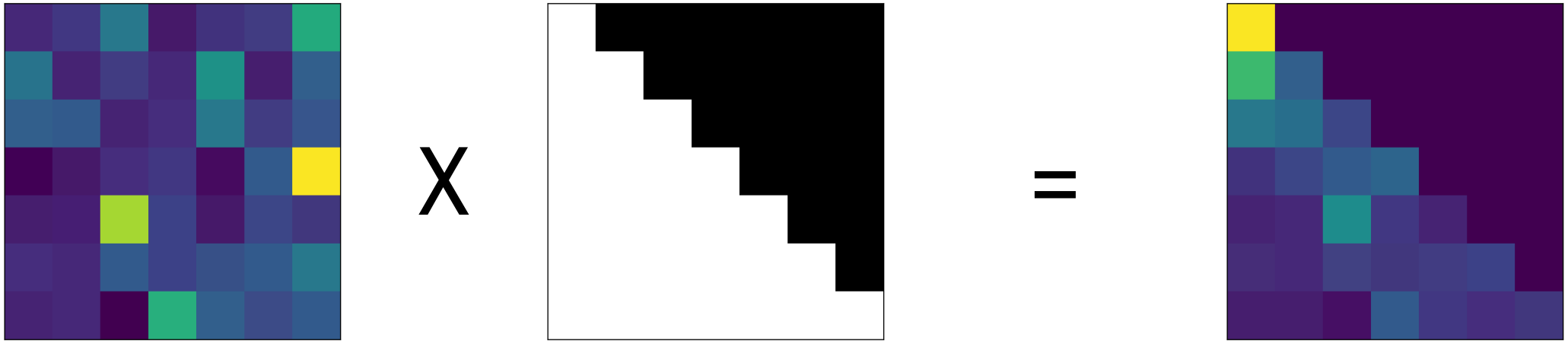
Decoders

- Models that are designed to **generate text**
- Next-word predictors:

$$P(w_i \,|(w_j)_{j \neq i}) = P_\theta(w_i | w_1 \ldots w_{i-1})$$

- **Problem**: How do we impede self-attention to consider future tokens?

Decoders

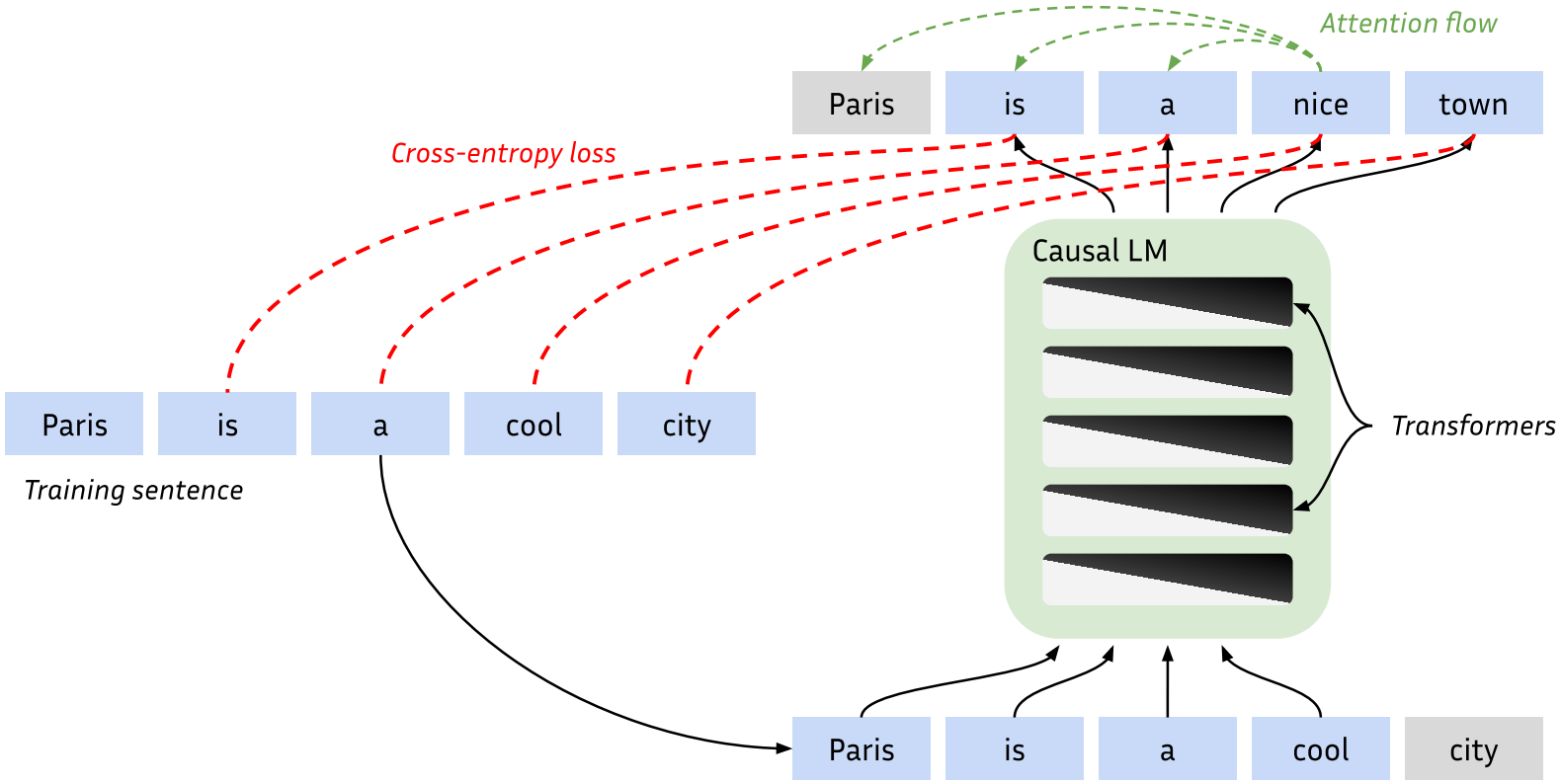


Attention *Attention mask* ***Causal** attention*

- Each attention input can only attend to previous positions

Decoders

- Teacher-forcing



Decoders

- What we have : a good model for $P_\theta(w_i|w_1 \dots w_{i-1})$
- What we want at inference:

$$W^* = \arg \max_{n, w_i \dots w_n} P_\theta(w_i \dots w_n | w_1 \dots w_{i-1})$$

- For a given completion length n, there are $|V|^n$ possibilities
  - e.g.: 19 new tokens with a vocab of 30000 tokens > #atoms in Ω
- We need approximations

Decoders

Greedy inference

- Keep best word at each step and start again:

$$W^* = \arg \max_{n, w_{i+1} \ldots w_n} P_\theta(w_{i+1} \ldots w_n | w_1 \ldots w_{i-1} w_i^*)$$

where $w_i^* = \arg \max_{w_i} P_\theta(w_i | w_1 \ldots w_{i-1})$

Decoders

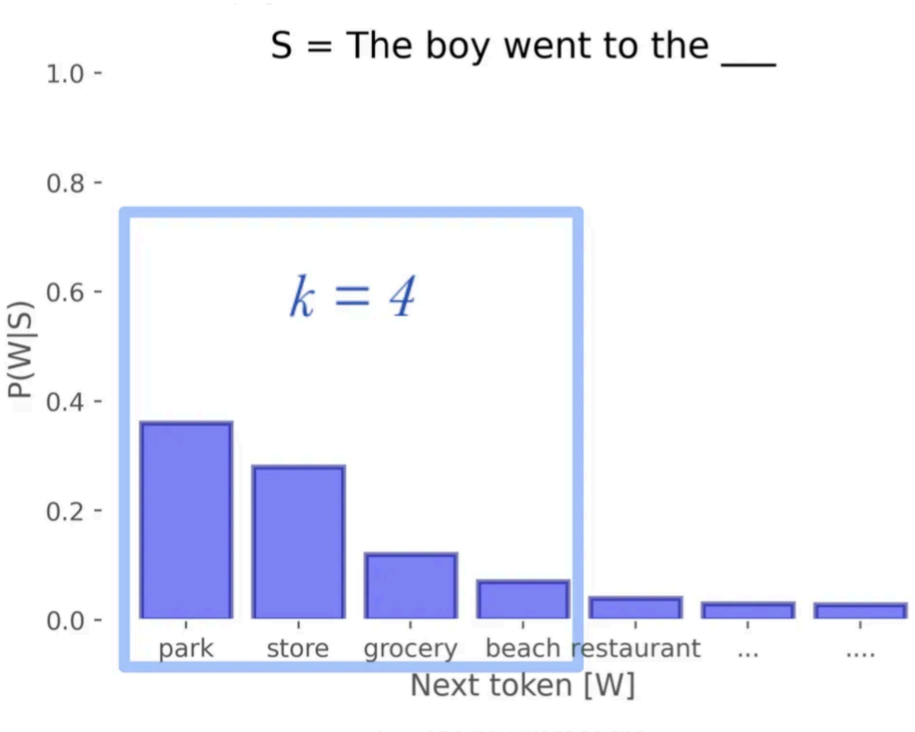
Beam search

- Keep best k chains of tokens at each step:
  - Take k best w_i and compute $P_\theta(w_{i+1}|\dots w_i)$ for each
  - Take k best w_{i+1} in each sub-case (now we have $k \times k$ (w_i, w_{i+1}) pairs to consider)
  - Consider only the k more likely (w_i, w_{i+1}) pairs
  - Compute $P_\theta(w_{i+2}|\dots w_i w_{i+1})$ for the k candidates
  - and so on...

Decoders

Top-k sampling

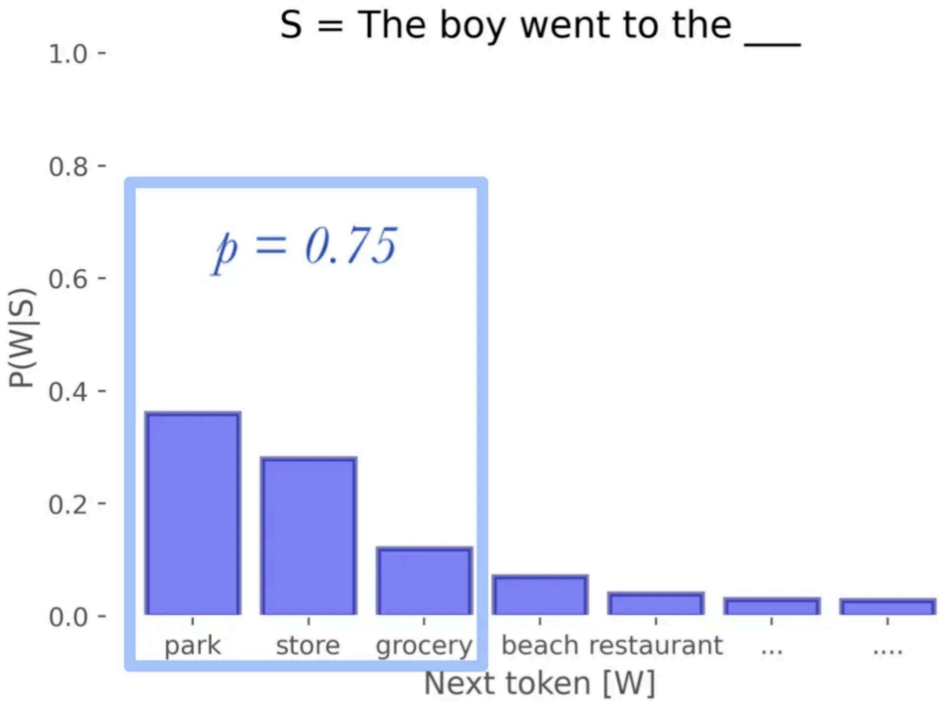
- Randomly sample among top-k tokens based on P_θ



Decoders

Top-p (=Nucleus) sampling

- Randomly sample based on P_θ up to $p\%$

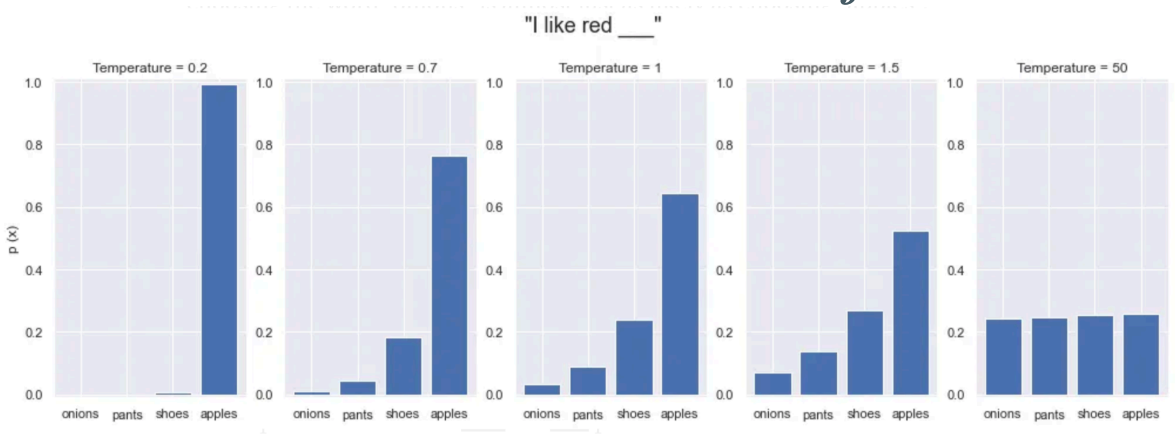


Decoders

Generation Temperature

- Alter the softmax function:

$$softmax_\tau(x) = \frac{e^{\frac{x_i}{\tau}}}{\sum_j e^{\frac{x_j}{\tau}}}$$



Decoders

- Encoders are mostly used to get contextual embeddings
  - They can also generate: T_{enc}("I love [MASK]")
- Decoders are mostly used for language generation
  - They can also give contextual embeddings : T_{dec}("I love music!")
  - Or solve any task using prompts:
    - "What is the emotion in this tweet? Tweet: '...' Answer:"
- Encoders-decoders are used for language in-filling

Decoders

- A useful evaluation metric: ***Perplexity***
- Defined as:

$$ppl(T_\theta; w_1 \dots w_n) = \exp\left(-\frac{1}{n}\sum_{t=1}^{n} \log P_\theta(w_t|w_{<t})\right)$$

Decoders

Zero-shot evaluation

- Never-seen problems/data
- Example: *"What is the capital of Italy? Answer:"*
  - Open-ended: Let the model continue the sentence and check exact match
  - Ranking: Get next-word likelihood for *"Rome"*, *"Paris"*, *"London"*, and check if *"Rome"* is best
  - Perplexity: Compute perplexity of *"Rome"* and compare with other models

Decoders

Few-shot evaluation / In-context learning

- Never-seen problems/data
- Example: *"Paris is the capital of France. London is the capital of the UK. Rome is the capital of"*
- Chain-of-Thought (CoT) examples:
  - Normal: *"(2+3)x5=25. What's (3+4)x2?"*
  - CoT: *"To solve (2+3)x5, we first compute (2+3) = 5 and then multiply (2+3)x5=5x5=25. What's (3+4)x2?"*

Lab session